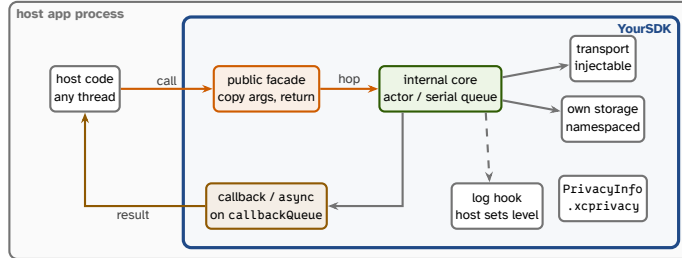


In one line: An SDK is a **guest with a contract**: a **small public surface** that is hard to misuse, **documented threading** (callbacks on a known queue), **no crash, no block, no launch cost**, errors as values, and a **compatibility promise** — SemVer at the source level, **library evolution** at the binary level — enforced by an API-diff check in CI. Every public symbol is forever; shipping it is the easy part, removing it takes a major version.

Inside the host app — the boundary



never: crash · block the caller · work at launch · swizzle by default · log secrets · call back while holding a lock

Binary-compatible? (resilient Swift library)

change in v2.x	old apps keep working?
add a method / type / conformance	yes
add a stored property to a non-@frozen struct	yes (layout is hidden)
add a case to a non-@frozen enum	yes; callers need @unknown default
add a protocol requirement with a default	yes
stored → computed property	yes
change an @inlinable body	old body still inlined in old apps
requirement without default · rename · remove	no — major version
add or remove @frozen	no

BUILD_LIBRARY_FOR_DISTRIBUTION=YES → library evolution + a .swiftinterface (module stability); cost: resilient values are accessed indirectly. Apple platforms only.

The surface

- **One entry type**, value-type **configuration** with safe defaults, required values in `init` (not setters you can forget). Allow instances; a shared convenience is optional — a forced singleton blocks tests and multi-account hosts.
- **Hard to misuse**: final classes (public, not open), enums over flag combos, tagged IDs, some `P` returns so the concrete type can change, internal by default. Kotlin: **explicit API mode**, no data classes in the public API (copy/constructor break on every new property).
- **Init**: cheap, no I/O, no network; nothing in `+load` / static initialisers; no swizzling unless the host opts in. Start work lazily, after launch.
- **Threading**: “callable from any thread” stated in the doc comment; state behind an actor / serial queue; public types `Sendable`; callbacks on a **documented queue** (main by default, or a `callbackQueue` like `URLSession`'s `delegateQueue`); never call back synchronously inside the call.
- **Errors**: one domain enum (non-@frozen, so new cases are additive) + underlying error; invalid config → `assertionFailure` in debug, an error in release. Retries and backoff inside, bounded.

Example — a surface that is hard to misuse

```
public struct PayConfig: Sendable {
    public let apiKey: String // required: in init
    public var environment: Environment = .production
    public var logLevel: LogLevel = .error // silent by default
    public var callbackQueue: DispatchQueue = .main
    public init(apiKey: String) { self.apiKey = apiKey }
}

public enum PayError: Error, Sendable { // NOT @frozen
    case invalidRequest(String), declined(code: String)
    case transport(underlying: any Error)
}

public final class PayClient: Sendable {
    public init(config: PayConfig) // no I/O here
    public func pay(_ r: PayRequest) async throws -> Receipt
    public func pay(_ r: PayRequest, completion: @escaping @Sendable
        (Result<Receipt, PayError>) -> Void) // on callbackQueue
    @available(*, deprecated, renamed: "pay(_:)")
    public func charge(_ r: PayRequest) async throws -> Receipt
}
```

Versioning — the promise

- **SemVer**: patch = fix, minor = additive, **major** = anything a caller must change. Deprecate first (`@available(*, deprecated, renamed:)`; Kotlin `@Deprecated` WARNING → ERROR → HIDDEN over several minors), remove at the next major.
- **API-diff in CI**: swift package `diagnose-api-breaking-changes` <tag>; Kotlin `binary-compatibility-validator` `apiDump` / `apiCheck` (in the Kotlin Gradle plugin from 2.2.0); a committed public-API file makes every surface change a reviewed diff.
- **Support window**: minimum OS, Xcode/Swift, Kotlin, RN versions written down; raising one is a **major**. Old SDK versions live in shipped apps for years.
- **Fleet visibility**: send the SDK version with every request (a header or `User-Agent`) — adoption per version, and the backend can **disable a broken version** remotely (kill switch) while hosts update.

Distribution

SPM	source package, or <code>.binaryTarget(url:checksum:)</code> pointing at a zipped XCFramework
XCFramework	archive per destination (<code>generic/platform=iOS,iOS Simulator</code> ; Skip Install = No, Build Libraries for Distribution = Yes) → <code>xcodebuild -create-xcframework</code> ; sign it
CocoaPods	trunk goes read-only on 2 Dec 2026 — existing builds keep working, no new versions; ship SPM
Android	AAR on Maven Central via the Central Publisher Portal (OSSRH ended 30 Jun 2025); ship <code>consumer-rules.pro</code> for R8
npm / RN	"exports", <code>react/react-native</code> as peer deps, native code autolinked (or an Expo config plugin)

Size, startup, privacy — the host pays

- **Budget** the SDK: measure the app-size and launch-time *delta* in a sample app per release. Few dependencies — a bundled copy of a popular library clashes with the host's (duplicate classes / symbols); vendor + rename, or depend on nothing.
- **Static XCFramework**: no dyld cost; **dynamic**: one copy for app + extensions.
- **Privacy manifest**: the SDK ships its own `PrivacyInfo.xcprivacy` (data collected, tracking domains, required-reason APIs). SDKs on Apple's commonly-used list (Alamofire, Firebase*, SDWebImage...) need the manifest **and a signature** when shipped as binaries; Xcode checks a new version is signed by the **same developer**.
- **Logging**: default silent or `Logger(subsystem: "com.example.sdk")`; a host-settable level and handler; never tokens or PII.

Testing and docs

- Internal seams (Transport, Clock, Storage) for your tests; ship a **test double** (`PayClient.mock`) so hosts can test *their* code.
- A **sample app** in the repo = the integration test CI builds against the *binary* artifact (catches a missing `.swiftinterface`, resource bundle or privacy manifest) on the minimum OS.
- DocC / KDoc for every public symbol incl. threading and error behaviour; a migration guide per major.

Traps

- ! Library evolution “for safety” in a source-only package — it is for *binaries* built separately from clients; it costs indirection.
- ! `@frozen` or `@inlinable` on day one — you can never add a stored property or change that body for old apps.
- ! Typed throws(`PayError`) is a promise too — the error type is now public API.
- ! Callback on “whatever thread the network used” — host UI updates off-main.
- ! Doing work in `init` or at launch “to be ready” — you just slowed every host.

Remember

Small, final, Sendable · config in init · callbacks on a known queue · errors as values · additive minors · API diff in CI · the host pays for your size, launch and privacy.

Related: system-design-sdk-messenger (analytics SDK pipeline) · api-design (SemVer, API guidelines) · modularization-spm · linking-and-launch-time · app-hardening-privacy (privacy manifest) · swift-idiom-patterns (tagged IDs)